

Client Permissions and Role-Gated UI

Overview

The Android client applies role-aware UI visibility through `UserPermissions`. This is a convenience and UX layer, not the source of truth for security. Backend enforcement still needs to hold the real boundary.

Current implementation

- `UserPermissions`
 - normalizes role names
 - decides access to moderation, economy tools, ads, livestream start, role assignment, suspension, analytics, and verification actions
- `TokenManager`
 - stores role-related user metadata locally
- role-aware screens include:
 - `SettingsActivity`
 - `LiveStreamsFragment`
 - `MainActivity`
 - moderation and admin surfaces

Important capabilities gated on the client

- livestream start visibility
- moderation dashboard visibility
- ad creation visibility
- user role dashboard visibility
- economy/admin-related navigation

Known issues

- frontend permissions are broader and more expressive in some areas than backend permission models
- role naming drift can cause visibility mismatches
- UI gating can hide or expose actions inconsistently if token/profile snapshots are stale

Scaling concerns

- every new staff or public role increases the normalization matrix
- duplicate permission logic across frontend and backend raises divergence risk
- stale local role data can create confusing UX after role changes

Future improvements

1. align frontend role helpers with a server-owned permission manifest
2. prefer capability flags over raw role-name branching where possible
3. refresh local role state more predictably after high-impact admin actions